

local-ai-project — Windowsマシンへの引き継ぎ手順書

対象: Mac mini (M4 Pro / macOS) で構築した `local-ai-project` を、Windows マシンで同じように動かすための移行ドキュメント。前提知識: ターミナル操作の基礎、`local-ai-project` の構成 (Ollama + Zed + GitHub + Python/Tkinter アプリ)。

0. 最初に理解すべきこと — 何が「運ばれ」、何が「やり直し」か

引き継ぎで一番つまずくのは、「全部がクラウドに乗っている」と誤解する点です。実際はこう分かります。

対象	引き継ぎ方法	Win側での作業
アプリのコード (.py)	GitHubリポジトリに入っている	<code>git clone</code> で取得
RAG用データ (.txt など)	同上 (コミットしていれば)	clone で一緒に来る
Zedの設定 (settings.json)	手順書内に全文あり (後述)	手動で貼り直し
Ollamaのモデル本体	✗ どこにも含まれない	Win側で再ダウンロード (必須)
Python本体・ライブラリ	✗ 含まれない	Win側で再インストール
Ollama本体・Zed本体	✗ 含まれない	Win側で再インストール

最重要ポイント: Ollamaのモデル (`qwen3.5-9b-16k` 等) は数GB～十数GBのバイナリで、GitHub にもGoogleドライブにも入っていません (~/.ollama にローカル保存されているだけ)。Windows 側で改めて取得する必要があります。ここを見落とすと「コードは動くのにAIが応答しない」で詰まります。

⚠ なぜGitHub経由で運ぶのか (Googleドライブ同期ではなく) `.git` フォルダをGoogleドライブでMac↔Win間同期すると、隠しファイルの競合でリポジトリが壊れることがあります。引き継ぎの正解はGitHub経由 (`git clone`)。Googleドライブは「成果物の閲覧・バックアップ」用と割り切るのが安全です。

1. Windows側の前提確認

作業を始める前に、Windowsマシンで以下を確認します。

- OS: Windows 10 (version 1809以降) または Windows 11、64bit
- GPU: DirectX 11対応 (2012年以降のPCならほぼ対応) ※Zedの要件
- ターミナル: **PowerShell** (スタートボタン右クリック → 「ターミナル」または「Windows PowerShell」)
- 管理者権限: 一部インストールで必要になる場合あり

Macでは「ターミナル (zsh)」でしたが、Windowsでは基本 **PowerShell** を使います。コマンドの大半は共通ですが、パスの書き方 (/ ではなく \、ドライブレター C: など) が異なります。

2. Ollama (AIエンジン) の導入とモデル再取得

2-1. Ollamaのインストール

ブラウザで ollama.com を開き、Windows版インストーラー (**OllamaSetup.exe**) をダウンロードして実行します。インストール後、Ollamaはバックグラウンドサービスとして常駐します (Macと同じく <http://localhost:11434> で待ち受け)。

PowerShellで動作確認:

```
None
ollama --version
ollama list
```

ollama list は最初は空のはずです (モデル未取得のため)。

2-2. モデルを再取得する

Mac側で使っていたモデルを、Windows側で **ollama pull** し直します。Mac側で **ollama list** を実行して名前を控えておき、同じ名前を取得します。

```
None
# 例: レジストリから取得できる標準モデル
ollama pull qwen2.5-coder:7b
```

```
ollama pull bge-m3
```

⚠ カスタムモデル名の注意(重要) `qwen3.5-9b-16k:latest` のような独自のタグ名は、`ollama create` (Modelfile) で自作したものである可能性があります。この場合、名前を指定して `pull` しても取得できません。

- 自作モデルの場合: Mac側の **Modelfile** (`ollama show モデル名 --modelfile` で内容を確認できる) をコピーし、Win側で `ollama create モデル名 -f Modelfile` で再作成する。
- ベースが公式モデルなら: 公式の同等モデル (例 `qwen2.5-coder:7b`) を取得し、アプリ側の参照モデル名をそれに合わせる方が早い。

2-3. 取得確認

```
None  
ollama list
```

アプリ/Zedが参照するモデル名が一覧に出ていればOKです。

3. Git(履歴管理)とプロジェクト本体の取得

3-1. Git for Windows のインストール

git-scm.com からWindows版Gitをインストールします。インストーラーの選択肢は基本デフォルトで問題ありません。確認:

```
None  
git --version
```

3-2. 作業フォルダを決めて clone する

Macでは「Googleドライブのマイドライブ内」に置いていましたが、**Windows**では`.git`同期トラブルを避けるため、ローカルの通常フォルダに**clone**するのを推奨します(例:`C:\dev`)。

```
None  
# 作業用の親フォルダを作って移動
```

```
mkdir C:\dev
cd C:\dev

# GitHubからプロジェクトを取得（自分のリポジトリURLに置き換え）
git clone https://github.com/TK20260401/local-ai-project.git

# 中に入る
cd local-ai-project
```

これで、コード・RAG用テキスト・READMEなどコミット済みのものが一式Win側に揃います。

3-3. 記録者情報の登録（この端末で初めてGitを使う場合）

```
None
git config --global user.name "あなたの名前"
git config --global user.email "あなたの会社メール"
```

3-4. 日々の運用（Macと同じ3コマンド）

```
None
git add .
git commit -m "作業内容のメモ"
git push
```

Mac↔Win間は、この `push` / `pull` で同期します。片方で `push` したら、もう片方で `git pull` してから作業を始めるのが鉄則。これで `.git` 破損の心配なく両マシンを行き来できます。

4. Python（自作アプリ）環境の再構築

4-1. Pythonのインストール

python.org からWindows版をダウンロードし、インストーラー起動時に「**Add python.exe to PATH**」に必ずチェックを入れます（これを忘れると `python` コマンドが通りません）。TkinterはこのインストーラーにGUI標準ライブラリとして同梱されています。

確認：

```
None  
python --version
```

4-2. 依存ライブラリの導入

アプリがOllamaのAPIを叩くために使っているライブラリ(例: `requests`)を入れます。リポジトリに `requirements.txt` があればそれを使います。

```
None  
# requirements.txt がある場合  
pip install -r requirements.txt  
  
# 無い場合 (最低限)  
pip install requests
```

4-3. アプリ起動テスト

```
None  
python アプリファイル名.py
```

Tkinterのウィンドウが立ち上がり、モデルが応答すれば成功です。

 応答しない場合、まずOllamaが動いているか(`ollama list` が通るか)、アプリ内の参照モデル名がWin側の `ollama list` の名前と一致しているかを確認。接続先は Mac と同じ `http://localhost:11434(=127.0.0.1:11434)` です。

5. Zed(エディタ)の導入と設定

Zedは2025年10月以降、Windowsで正式・安定サポートされています(DirectX 11レンダリング、weekly更新、Ollama連携対応)。

5-1. Zedのインストール

PowerShellから `winget` で入れるのが最も簡単です。

```
None  
winget install Zed.Zed
```

または zed.dev からWindows版インストーラー(.exe)を落として実行。インストーラーがPATHを自動設定するので、後からターミナルで `zed` が使えます。

確認:

```
None
zed --version
```

5-2. プロジェクトを開く

```
None
cd C:\dev\local-ai-project
zed .
```

初回起動時に信頼性確認(Unrecognized Project)が出たら「Trust and Continue」を選択。

5-3. Ollama接続設定(settings.json)

`cmd + ,` ではなく、Windowsでは **Ctrl + ,** (カンマ) で設定ファイル(`settings.json`)を開きます。以下を貼り付けて **Ctrl + s** で保存。

```
JSON
{
  "theme": "One Dark",
  "assistant": {
    "version": "2",
    "provider": {
      "name": "ollama",
      "api_url": "http://localhost:11434"
    },
    "default_model": "qwen2.5-coder:7b"
  }
}
```

`default_model` は、Win側の `ollama list` に実在する名前を入れること。Macの `qwen3.5-9b-16k:latest` をそのまま書いても、Win側に無ければ繋がりません。

5-4. AIチャットパネルを開く

Windowsでは **Ctrl + Ctrl + a** ではなく、メニューバーの「View → Toggle Assistant Panel」から開くのが確実です(ショートカットはZedのバージョンで変わることがあるため、コマンドパレット **Ctrl + Shift + P** →「assistant」で検索してもよい)。

6. Mac ↔ Windows 操作の対応表 (早見)

項目	Mac (zsh)	Windows (PowerShell)
ターミナル	ターミナル.app	PowerShell / ターミナル
ホームからのパス	<code>~/Library/CloudStorage / ...</code>	<code>C:\Users\ユーザー名\...</code>
パス区切り	<code>/</code>	<code>\</code> (PowerShellは <code>/</code> も可)
設定を開く (Zed)	<code>cmd + ,</code>	<code>Ctrl + ,</code>
保存 (Zed)	<code>cmd + s</code>	<code>Ctrl + s</code>
Ollama接続先	<code>http://localhost:11434</code>	<code>http://localhost:11434</code> (同じ)
Ollamaインストール	公式pkg / brew	公式 <code>OllamaSetup.exe</code>
Zedインストール	<code>brew install --cask zed</code>	<code>winget install Zed.Zed</code>
プロジェクト配置	Googleドライブ内	<code>C:\dev</code> (ローカル推奨)

7. 完全オフライン動作の確認 (Winでも同じ)

Macと同様、Windowsでもネットを切って動作確認します。

1. Wi-Fi／有線をオフにして完全オフライン化
2. Ollamaが常駐しているか確認 (タスクトレイのOllamaアイコン、または `ollama list`)
3. アプリを起動 → `http://localhost:11434` へ接続してストリーミング応答が返るか
4. Zedのアシスタントパネルからローカルモデルに話しかけて応答が返るか

すべてローカル (`127.0.0.1`) で完結するため、ネット遮断でも動けば、Win側でもオフライン環境の再現が完了です。

8. 引き継ぎ完了チェックリスト

- ☐ Ollamaをインストールし、必要モデルを `ollama pull / ollama create` で再取得した
- ☐ `ollama list` に、アプリとZedが参照するモデル名が出ている
- ☐ Git for Windowsを入れ、`git clone` でプロジェクトを取得した
- ☐ Python(PATH追加)を入れ、`pip install` で依存を入れた
- ☐ アプリが起動し、モデルが応答した
- ☐ Zedをインストールし、`settings.json` のモデル名をWin側に合わせた
- ☐ Zedのアシスタントパネルからローカルモデルに接続できた
- ☐ Wi-Fiオフの完全オフラインで一連が動いた
- ☐ `git pull / git push` でMacと同期できることを確認した

付録:トラブル早見

症状	原因	対処
<code>python</code> が見つからない	インストール時にPATH未追加	Pythonを再インストールし「Add to PATH」にチェック
AIが無応答	Ollama未起動 or モデル名不一致	<code>ollama list</code> を確認、参照モデル名を一致させる
<code>zed</code> コマンドが通らない	PATH未反映	ターミナルを開き直す／PC再起動、または <code>winget</code> で入れ直す
clone後に日本語ファイル名が化ける	Gitの文字コード設定	<code>git config --global core.quotePath false</code>
モデル名で <code>pull</code> できない	自作 (Modelfile) モデル	Mac側の Modelfile を持ち込み <code>ollama create</code> で再作成
リポジトリが壊れた	<code>.git</code> をドライブ同期していた	ローカルに <code>git clone</code> し直す (Drive同期はしない)